GradeMaster

Academic Grade Calculator & Target Planning System

Midterm Project Report
Author: **Dheeraj Patil**
Date: **November 1, 2026**
GitHub: DheerajsPatil/grade-calculator

01 What You Made

GradeMaster is a modern, responsive single-page web application engineered to solve the everyday challenges students and educators face when tracking course standings, computing weighted percentages, planning final exam targets, and monitoring semester GPA.

⚖️ Weighted Category Engine

Supports syllabus structures where course components have differing percentage weights (e.g., 20% Midterms, 30% Homework, 50% Final). It dynamically validates total weight allocation (alerting at ≠100%) and features a toggle for direct percentage inputs or fractional points earned (e.g., 45/50).

🎯 Final Exam Target Planner

Solves the inverse linear equation: `Target = Current × (1 - W) + Exam × W` to compute the exact score required on a comprehensive final exam. Provides qualitative advice badges (e.g., "Secured", "Achievable", "Challenging", or "Extra Credit Required").

📊 Points-Based Grade Calculator

Designed for courses graded on cumulative points (e.g., 850 total points over a semester). Computes live totals for earned points, possible points, raw percentage scores, and equivalent letter grades.

🎓 Semester GPA Calculator

Calculates semester Grade Point Average on a standard 4.0 scale by multiplying course credit hours with grade points to yield total Quality Points (QP) and cumulative GPA.

✨ Interactive Visualizations & Architectural Highlights

The application is built on a **zero-dependency architecture** (vanilla HTML5, CSS3, and ES6+ JavaScript), guaranteeing lightning-fast startup and total offline availability. It features an **animated SVG circular progress gauge** showing real-time grade standings, full **Dark/Light mode** theming with CSS custom properties, pre-built **syllabus presets** (Standard College, STEM Lab, High School), an interactive **Custom Grading Scale** modal, and automatic `localStorage` persistence.

02 Who It Is For

Academic grading systems vary wildly across institutions, departments, and course levels. GradeMaster is targeted toward four key user groups:

👩‍🎓 College & University Students

Undergraduate and graduate students managing diverse syllabi with complex weighting, drop-lowest policies, and high-stakes final exams. GradeMaster eliminates guesswork during finals week.

👛 High School & AP/IB Students

Students aiming for selective university admissions who must closely monitor weighted GPAs, test scores, and assignment point totals to maintain honors eligibility.

👩‍🏫 Academic Advisors & Counselors

Advisors conducting student intervention sessions. They can load syllabus templates to model academic recovery scenarios and map realistic paths to good academic standing.

👩‍🏫 Educators & Teaching Assistants

Instructors who need a lightweight tool to verify grade distribution models, test boundary conditions, or demonstrate grading policies to students.

User Persona Illustration (Maya, Sophomore Computer Science Major):

"It's two weeks before finals. Maya has an 84.5% in Data Structures. Her final exam counts for 25% of her course grade, and she needs an A- (90%) for her scholarship. Using the Final Exam Target Planner, she immediately learns she needs a 106.5% on the final exam. Recognizing that an A- requires extra credit, she switches her target to a solid B+ (87%), discovering she needs a realistic 94.5% on the exam, allowing her to optimize her study hours effectively."

03 How You Made It With AI

The project was developed through an advanced AI-assisted pair-programming workflow leveraging **Antigravity** powered by **Gemini 3.8 Flash**. The methodology followed an iterative, structured engineering lifecycle:

1. **Iterative Planning & Architectural Synthesis:** Rather than jumping directly to raw code, the development started in *Planning Mode*. The AI agent generated a detailed technical proposal (`implementation_plan.md`) defining module boundaries, state management models, SVG geometry formulas, and validation rubrics before writing application code.
2. **Component-Driven Code Generation:** The UI and calculation engine were split into clean concerns:
   - **Semantic HTML (`index.html`)**: Built with accessible form fields, ARIA roles, clean tab switching, and modal dialogs.
   - **Modular CSS (`styles.css`)**: Configured with CSS custom properties (variables) for seamless light/dark theme toggles, glassmorphic card elevations, responsive media breakpoints, and print-friendly styles.
   - **Reactive Engine (`app.js`)**: Implemented an event-delegated architecture with debounced `localStorage` synchronization and mathematical solvers.
3. **Mathematical Solver Formulation:** The AI formulated and implemented exact solutions for weighted category normalization when total syllabus weights are in-progress ($W_{total} < 100\%$) and the inverse final exam equation:
   `Score_{required} = (Target - Current × (1 - W_{exam})) / W_{exam}`
4. **AI-Driven Tooling & Git Automation:** The AI utilized background execution tools to initialize Git repositories, configure tracking branches, produce comprehensive project documentation (`README.md`, `.gitignore`), and seamlessly push code to GitHub using the GitHub CLI (`gh`).

04 What You Checked (Quality Assurance & Verification)

To ensure production-grade reliability, a rigorous automated and manual testing matrix was executed across calculations, DOM bindings, and boundary edge cases:

Module / Function	Test Input Vectors	Expected Value	Tested Result	Status
Weighted Grade	HW 20% @ 90, Midterm 30% @ 80, Final 50% @ 95	89.50%	89.50%	PASSED
Final Target Solver	Current 84.5%, Exam Weight 25%, Target 90.0%	106.50% (Extra Credit)	106.50%	PASSED
Points Aggregator	88/100, 45/50, 142/150, 92/100 (367 / 400 pts)	91.75% (A-)	91.75%	PASSED
Semester GPA	4 cr A- (3.7), 4 cr A (4.0), 3 cr B+ (3.3), 3 cr A (4.0)	3.76 GPA (52.7 / 14 QP)	3.76 GPA	PASSED
DOM Bindings	45 JavaScript Element IDs checked against HTML	100% matched	45 / 45 matched	PASSED

🛡️ Boundary & Edge Cases Tested

• **Zero Division:** Zero credits or 0 total points return safe fallbacks without throwing NaN or infinity.
• **Weight Over-allocation:** Automatically shifts progress bar from green to warning amber or danger red when weights exceed 100%.
• **Negative / Out-of-bounds:** Handles inputs > 100% (bonus credit) and negative values gracefully.

📱 Visual & Cross-Platform Verification

• **Responsive Design:** Tested across desktop (1200px), tablet (768px), and mobile viewports (375px).
• **Local Persistence:** Verified data preservation across browser reloads.
• **Print Media:** Tested `@media print` formatting for clean PDF export without navigational clutter.

05 What Is Next

While the current application is feature-complete, future development milestones will expand GradeMaster into an all-in-one academic planner:

📱 Progressive Web App (PWA)

Implement service worker caching and web app manifest so students can install GradeMaster directly on iOS and Android home screens and access it completely offline.

🔄 LMS Integration (Canvas / Blackboard)

Allow students to securely import their live assignment grades, syllabus weighting schemes, and quiz scores directly via Canvas REST API or CSV export.

🎲 Monte Carlo "What-If" Forecasting

Incorporate probabilistic simulation modeling where students can set minimum/maximum expected scores on upcoming exams and view percentage likelihoods of achieving specific letter grades.

☁️ Multi-Semester Cloud Synchronization

Enable encrypted cloud synchronization with Google Drive / OneDrive to track 4-year cumulative degree progress, major GPAs, and graduation honors qualifications.